

Note: We mentioned the Y gates are responsible for a phase as well as a bit flip. What does that mean? To display that effect, let's apply Y gate on the $|+\rangle$ state.

```
qc = QuantumCircuit(1)
qc.h(0)
qc.y(0)
qc.draw(output='mpl')
```

Execute the circuit on the backend and plot the output:

```
backend = Aer.get_backend('statevector_
simulator')
result = execute(qc, backend).result().
get_statevector(qc, decimals=3)
print (result)
plot_bloch_multivector(result)
```

Figure 3 shows the gate's placement and output statevector.

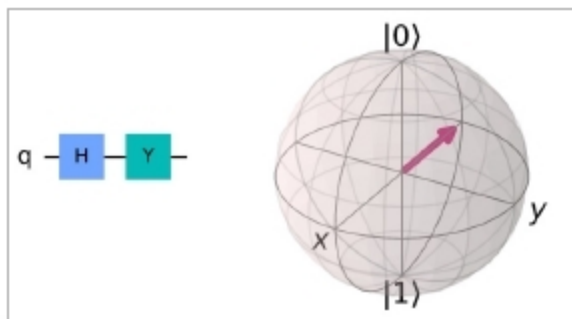


Figure 3: Displaying the output of a Y gate applied to circuit with H gate

When we print the statevector, the result can be found as $[0. - 0.70710678j, 0. + 0.70710678j]$, which means the phase has been flipped (recall H gate gives a *statevector* $[0.70710678 + 0.j, 0.70710678 + 0.j]$). Also, when you see the resultant Bloch sphere and compare it with that of H gate, we see that the bit has been flipped as well.

c. R_ϕ gates: The R_ϕ -gate is *parameterised*, that is, it needs a number (ϕ) to tell it exactly what to do. The R_ϕ -gate performs a rotation of ϕ around the Z-axis direction (and as such is sometimes also known as the R_z -gate). Its matrix is

given as follows:

$$R_x(\theta) = \begin{bmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} & -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix}$$

```
from numpy import pi
# Rx Gate
qc = QuantumCircuit(1)
qc.rx(pi/4,0)
qc.draw('mpl')
```

Execute the circuit and visualise the result. Figure 4 shows the circuit and

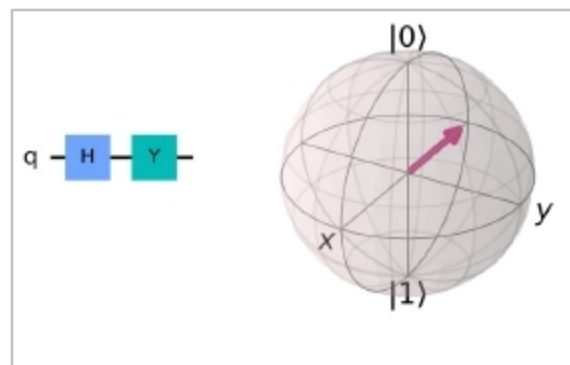


Figure 4: Rotation gate R_x and its result on Bloch sphere for $\phi = \frac{\pi}{4}$

resultant Bloch sphere.

```
backend = Aer.get_backend('statevector_
simulator')
result = execute(qc, backend).result().
get_statevector(qc, decimals=3)
print(result)
plot_bloch_multivector(result)
```

Result: $[0.92387953 + 0.j, 0. - 0.38268343j]$; i.e. the resultant state $|\psi\rangle = 0.924 |0\rangle - i0.383 |1\rangle$

d. U gates: This is a generic single-qubit rotation gate with three Euler angles. These are the most basic quantum gates. U stands for Universal, which means any other gate can be created with the help of a combination of U gates. Earlier versions of Qiskit had three varieties of U gates, i.e., U1, U2 and U3. U2 and U1 gates were specific cases of the U3 gate. For U2 gate- $\theta=\pi/2$, and for U1 gate- $\theta=\phi=0$, respectively. The use of U1, U2 and U3 has been deprecated from qiskit-terra version 0.16.0. Now, U gates can

be accessed directly by the name of 'u'. The general U gate is given by the following matrix:

$$U(\theta, \phi, \lambda) = \begin{bmatrix} \cos \frac{\theta}{2} & -e^{i\lambda} \sin \frac{\theta}{2} & e^{i\phi} \sin \frac{\theta}{2} & -e^{i(\phi+\lambda)} \cos \frac{\theta}{2} \end{bmatrix}$$

The following code snippet let's you use U gates:

```
qc = QuantumCircuit(1)
qc.u(2*pi/3, pi/2, 0, 0)
qc.draw(output='mpl')
```

Output on a Bloch vector can be visualised with the following code. Figure 5 shows the result.

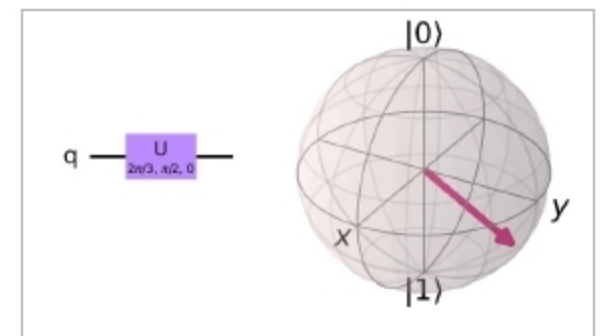


Figure 5: The $U(\frac{2\pi}{3}, \frac{\pi}{2}, 0)$ gate in single qubit circuit and its result plotted on Bloch sphere

```
backend = Aer.get_backend('statevector_
simulator')
result = execute(qc, backend).result().
get_statevector(qc, decimals=3)
print(result)
plot_bloch_multivector(result)
Result: [0.5 + 0.j, 5.30287619e-17 + 0.8660254j]
```

2. Two-qubit gates

These gates operate on two qubits.

a. CNOT gate: The CNOT (or cx) gate is an important two-qubit gate that plays an important role in establishing entanglement in quantum experiments. CNOT is short for Controlled-NOT gate. Its matrix is given by:

$$CX = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$